A Quick Primer on SMT Solving

Alec Critten

September 2025

Overview

1. SAT solving
2. SMT solving

Preliminaries

- Recall Boolean logic: $\mathbb{B} = \{true/\top, false/\bot\}$
- We want to determine if a formula in *propositional logic* (i.e. with Boolean terms) has any solutions so that it is true
  - A formula that is always true is a *tautology* (ex. A or not A)
  - A formula that has at least one solution is *satisfiable*
  - A formula with no solutions/always false is *unsatisfiable*
  - A formula that is never true is a *contradiction* (ex. A and not A)

What is a SAT solver?

- A SAT solver is a fast automated reasoning system to determine if a Boolean formula is satisfiable or not
- Returns `sat`, `unsat`, `unknown` (timeout)
- `sat` case: we can get a *model*/satisfying assignment
- `unsat` case: we can get a *unsat core* (the smallest bit of the formula that is unsat) and a *proof* of unsatisfiability
- (Note: the ETCS checker flips this around so a proof shows safety and sat means constraint violation)

SAT Algorithms/Proof Calculus

- Conflict Driven Clause Learning (CDCL)
  - Basic idea: learn from mistakes by adding clauses when backtracking
  - (The underlying mechanics of this are not important)
- Resolution
  - $(C \vee A) \wedge (\neg A \vee D) \Rightarrow (C \vee D)$
  - Idea: resolve clauses until you have the empty clause to show unsatisfiability

Propositional Logic examples

$(A \vee B) \wedge (B \vee C)$
SAT, model: $A \to \top, B \to \bot, C \to \top$

$(A \vee \neg B) \wedge (B \vee C) \wedge (\neg A \wedge \neg C)$
UNSAT

What is SMT?

Satisfiability Modulo Theories (SMT):

- generalises propositional satisfiability (SAT) to first-order logic
- uses decision procedures to check satisfiability with respect to background theories
  - Inbuilt theories: linear arithmetic, bit-vectors, arrays, etc.

Two main approaches for SMT solving

- Eager solving: convert the theory problem to SAT ("blasting")
- **Lazy solving:** use a dedicated theory procedure (next slide)

SMT's "lazy solving" approach

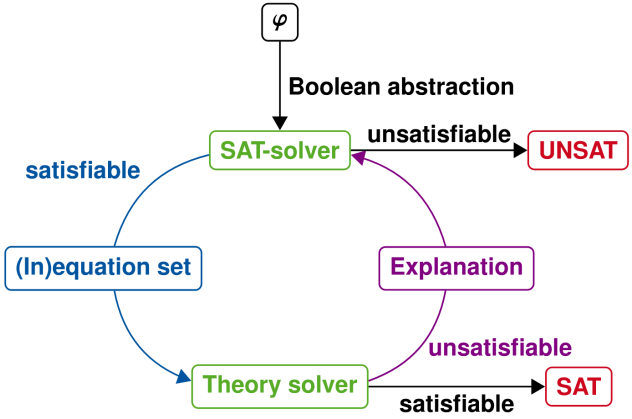


Figure: (Taken from Erika Ábrahám's lectures in Marktoberdorf)

The interplay between SAT and theory solvers

Suppose we have $\phi = (x < 3) \wedge (x > 4)$. Obviously there is no such x, but the solver deduces this way:

1. **Boolean abstraction**: First, the solver abstracts the first-order formula into propositional logic, so

$$\underbrace{(x < 3)}_{A} \wedge \underbrace{(x > 4)}_{B}$$

The interplay between SAT and theory solvers, cont.

2. **Boolean assignment**: The SAT solver makes a decision to set $A \rightarrow \top$ and $B \rightarrow \top$.

The interplay between SAT and theory solvers, cont.

3. **Theory symbols**: SMT solver observes $x < 3$ and $x > 4$ on a purely symbolic level, then determines they belong to the theory of integer arithmetic (LIA).

The interplay between SAT and theory solvers, cont.

4. **SAT modulo LIA**: Solver checks satisfiability of $A \wedge B$ w.r.t. LIA and returns `unsat`.

What can we express in SMT that we can't get in SAT?

We get first-order logic!

- Quantifiers: for all ($\forall$), exists ($\exists$)
- Arithmetic constraints, e.g. $(x > 1) \wedge (x < 5)$
- Functions, e.g. $f(x) = x + 1$
- Implication e.g. $A \Rightarrow B$
- Floating-point numbers, arrays, bit-vectors, etc.